

プログラミング A レポート（レポート課題 1）

提出者: 青木 優

学籍番号: 09B26001

担当教員: ヌリ オリビエ

提出年月日: 2026 年 7 月 16 日

1 課題内容

1. 挿入ソート、バブルソートを行うプログラムを作成せよ。
2. また、配布した選択ソートの時間計測版 (r1_selection-sort-2.c) をもとに、挿入ソート、バブルソートに対し実行時に `gen_data()` から配列に入力を与えられることができ、かつ、また時間を計測できるようなプログラムを作成せよ。なお、実行結果として、時間とソート済み配列両方を表示すること。
3. 配布した時間計測版選択ソートおよび 1. で作成した時間計測版のプログラムを用いて各ソートに要する実行時間を測定し、測定結果から言えることを考察せよ。なお、公平に比較するため、各ソートプログラムの入力として与える配列データは、`gen_data()` から同じものを使うこと。
4. (発展課題) 他のソート手法を調査し、プログラムを作成せよ。また、同様に実行時間を測定し、考察を行え。

2 アルゴリズム

フローチャートは分かりやすく C 言語でかかれたアルゴリズムをわざわざ図で書き直すことで分かりにくくする非本質要素なので除外します。コード見ろ！！

2.1 選択ソート

左右を比較して小さいものを左、大きいものを右という風に順番にやっていると当然今操作を行っている対象の中で最も小さいものが右端に移動するので、後は最も大きいものを右にはじき出す作業を配列の長さ-1 回分やるだけ。

2.2 挿入ソート

左から既にソートされている領域のどこに入るかを判定して挟み込んでいくだけの単純作業。

2.3 バブルソート

これ選択ソートと同じじゃね？やってることはじき出してるだけじゃんこれ。ゴミ。

2.4 クイックソート

どんどん 2 つに分けていって再帰で処理して最後 2 個まで分かれたら普通にでかいほう右にやって `return`。

2.5 マージソート

一旦最小単位 (1 個か 2 個の値) になるまで下がっていきそこからマージを繰り返していく方式。左右それぞれのブロックで整列されていることを前提にソートしていくのでトップダウンというよりボトムアップ的に組み立てる。

3 プログラムの設計

本節ではプログラムの設計について述べる。

3.1 入力形式の設計

本プログラムでは RANDOM(乱数)、PARTIAL_RANDOM(部分ソートされた配列)、REVERSE(反転ソート)、INPUT(手動入力) の各モード及び乱数生成時のデータサイズとそのシードを選択可能にした。

3.2 出力形式の設計

実行中の動作表示と csv 形式でのデータ出力を完備。

3.3 データ構造の設計

3.3.1 Data 構造体

ソート用のデータを管理する構造体

- data : 連続データの先頭を指すポインタ
- size : データの要素数を示す値

3.3.2 Mode

データ入力のモードを表す enum

3.3.3 Config 構造体

ソートするためのデータの形式を管理する構造体。

- mode : その名の通りモード
- seed : その名の通りランダムデータ生成時のシード
- data_size : その名の通りデータのサイズ
- ration : PARTIAL_SORT の時にどれくらいソートされているべきか表す割合

3.4 関数の設計

多すぎるので主要部分だけ解説

3.5 主部の説明

さっきやった

表 1: 関数の引数とその用途

関数名	用途	引数名	引数の用途
test_condition_with_data	ソートをする	SortAlgorithm data do_check show_data algorithm_name	使うソート関数 ソートするデータ ソート後の検証するか ソート前後のデータ表示するか ソート関数の関数名
test_condition	データ生成しソート	config	データを生成するための Config
write_csv	csv 書き込み	file_name time_data_count size_data time_data	書き込む csv ファイルの名前 時間データの個数 ソートしたデータの個数 時間データ本体

4 エ夫した点

貰ったヘッダファイルがあまりにもクソだったので自分で全部書き直した。具体的にはデータとサイズを構造体にまとめ、gen_data の設定も構造体にまとめ、free_data では開放したデータの再解放を防ぐために NULL に置換、ストップウォッチは一つの関数にまとめた上でグローバル変数の仕様を避けて static 変数を使った。ソート関数は関数ポインタで扱うことで処理の共通化を実現。出力データは csv にすることでグラフ化しやすいようにし、実行時間は ms 程度の精度しかないことに気づいたので整数値で管理。

5 プログラムの制限事項

本プログラムでは、データ型に int を用いたため他データのソートができない。また、入力値の検証をしていないので場合によっては未定義動作を引き起こす。

6 実行例

プログラムの動作を確認するために、様々な初期データを与えてプログラムを実行した。その結果、いずれの場合においてもソートが実現されていることを確認した。

20 個の初期データを与えてプログラムを実行した場合の実行結果の一例を下記に示す。

```
input data size: 20
input mode(0: RANDOM, 1: PARTIAL_RANDOM, 2: REVERSE, 3: INPUT): 3
generating test case...
input data
>> 22 34 41 15 51 90 44 99 23 10 13 45 32 60 45 80 0 1 9 98
test case is generated
```

```

mode: INPUT
data_size: 20

data before sorted
22 34 41 15 51 90 44 99 23 10 13 45 32 60 45 80 0 1 9 98
copying data...
copied!
start sorting with: selection_sort
sorting...
finished sorting!
    time(ms): 0

data after sorted
0 1 9 10 13 15 22 23 32 34 41 44 45 45 51 60 80 90 98 99
checking...
succeed!

data before sorted
22 34 41 15 51 90 44 99 23 10 13 45 32 60 45 80 0 1 9 98
copying data...
copied!
start sorting with: insertion_sort
sorting...
finished sorting!
    time(ms): 0

data after sorted
0 1 9 10 13 15 22 23 32 34 41 44 45 45 51 60 80 90 98 99
checking...
succeed!

data before sorted
22 34 41 15 51 90 44 99 23 10 13 45 32 60 45 80 0 1 9 98
copying data...
copied!
start sorting with: bubble_sort
sorting...
finished sorting!
    time(ms): 0

```

```
data after sorted
0 1 9 10 13 15 22 23 32 34 41 44 45 45 51 60 80 90 98 99
checking...
succeed!
```

```
data before sorted
22 34 41 15 51 90 44 99 23 10 13 45 32 60 45 80 0 1 9 98
copying data...
copied!
start sorting with: quick_sort
sorting...
finished sorting!
    time(ms): 0
```

```
data after sorted
0 1 9 10 13 15 22 23 32 34 41 44 45 45 51 60 80 90 98 99
checking...
succeed!
```

```
data before sorted
22 34 41 15 51 90 44 99 23 10 13 45 32 60 45 80 0 1 9 98
copying data...
copied!
start sorting with: merge_sort
sorting...
finished sorting!
    time(ms): 0
```

```
data after sorted
0 1 9 10 13 15 22 23 32 34 41 44 45 45 51 60 80 90 98 99
checking...
succeed!
```

7 考察

7.1 RANDOM

配列の要素の数 n がソートの性能にどのような影響を与えるかを確認するため、配列の要素の数 n を 1 から 65536 までさまざまに変え、クイックソート、マージソートを除くソートに要する実行時間を計測した。配列の各要素の初期値としては乱数を用いた。計測に用いたコンピュータの諸元を表 2 に示す。以降の結果は

10 種類の乱数シードを用いた結果の平均である。なお、評価に用いたプログラムを付録に示す。

配列の要素数に対する選択ソート、挿入ソート、バブルソートの実行時間を図 1 及び 2 に示す。図 1 は結果を通常グラフで、図 2 は結果を両対数グラフで表記したものである。図のように両対数グラフで直線の傾向を示すことから、配列の要素数と実行時間にべき関数の関係があることが読み取れる。この理由は以下のとおりである。まず、2 節で述べた選択ソートのアルゴリズムの中で、比較的負荷の高い処理は配列の中身を比較する比較処理と、配列の中身を交換する交換処理である。比較処理は、1 ステップ目では $n - 1$ 回、2 ステップ目では $n - 2$ 回、 $\dots$ 、 $n - 1$ ステップ目では 1 回行われる。そのため比較処理の実行回数は、 $(n - 1) + (n - 2) + \dots + 1$ 、すなわち、 $n(n - 1)/2$ 回である。一方、交換処理は各ステップで 1 回ずつ行われるので、 $n - 1$ 回である。また、バブルソートや挿入ソートでも同等のオーダーとなっていることからこれらのアルゴリズムは時間計算量 $O(n^2)$ で処理することが分かる。

次にクイックソートは面倒なのでマージソートについて、特に $n = 2^m$ のときについて考える。ボトムアップ式に考えて一番下の段から考えていくことにする。段が k 段目のとき、既にソートされた 2^{k-1} 個のデータ同士をマージする操作をそれぞれのブロックに対してすることになるが、結局これは全部の要素に対してやるので必要な計算量はおおよそ $O(2^m)$ 回である。あとはこれが $1 \leq k \leq m$ について足し合わせて結局の計算量は $O(m2^m)$ くらい、すなわち $O(n \log n)$ である。

グラフが線形に見えるのは n が小さすぎるせいですねはい。

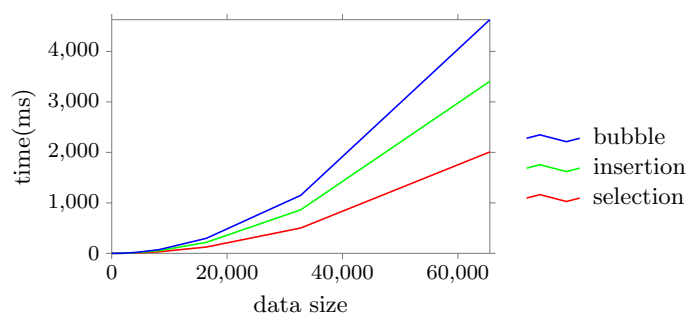


図 1: 1 から 65536 個の要素に対するソートの実行時間

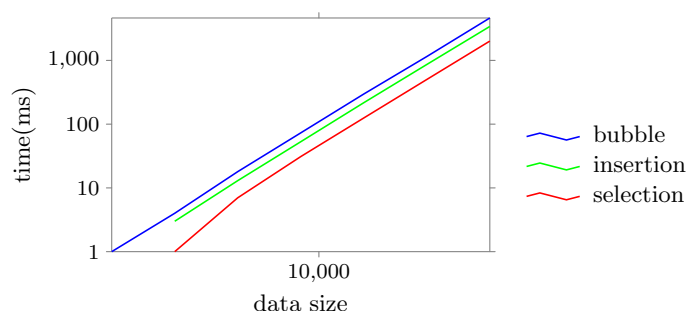


図 2: 1 から 65536 個の要素に対するソートの実行時間

また、マージソート及びクイックソートに対しては 1 から 67108864 個の要素に対してソートを実行し同様に時間を測定した。

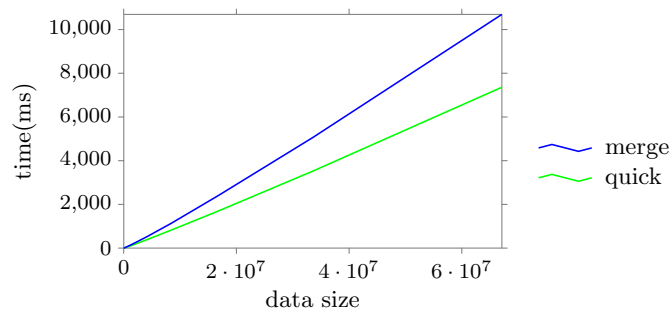


図 3: マージソート、クイックソートによるソートの実行時間

7.2 REVERSE

また、同様にして mode を REVERSE にした場合のソートも実行した。

図 4 を見るとバブルソート、挿入ソートは順当に遅くなっているが選択ソートはほとんど時間が変わっていない。これは選択ソートは毎回の走査でソートされていない部分の全ての要素を見なければならず、配列が逆になっていようが大して変わらないからだと思われる。次に、クイックソートとマージソートで逆に早くなっている理由は、逆ソートされた配列の作り方に問題があり、両方とも左右に分割してその左右をそれぞれソートしていくアルゴリズムだが、これは左右分割がちょうど結果として出てくる配列の中央と同じになっている時が最高効率である。そして逆ソート配列は配列の大きさ n にたいして 1 ずつ小さくなるように作られているのでこの最高効率の配列が常に渡されるということだ。よってとても速い。

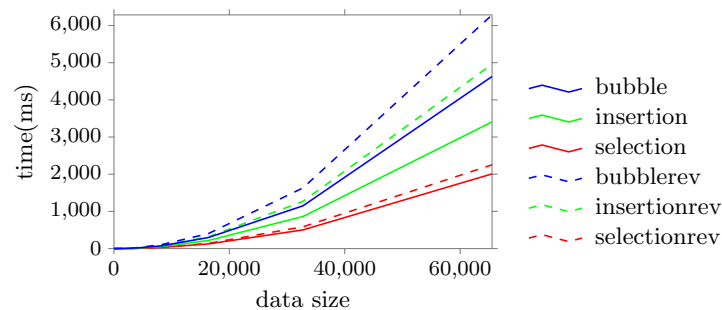


図 4: 1 から 65536 個の要素に対するソートの実行時間

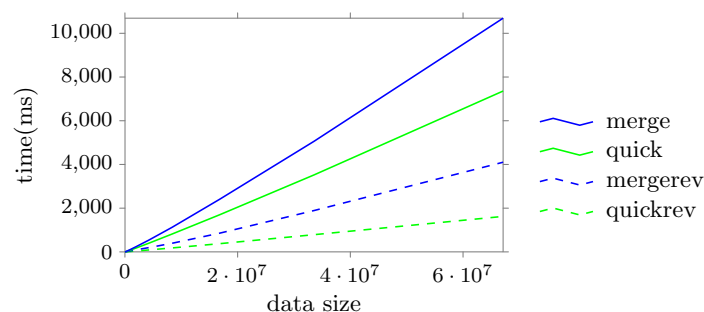


図 5: マージソート、クイックソートによる逆転ソートの実行時間

表 2: 性能評価に用いた環境

CPU	Ryzen 7 3.8 GHz
Memory	4 GB
OS	Ubuntu 24.04(WSL)
C コンパイラ	gcc

8 感想

謎のこだわりを発揮し要らない所まで抽象化したりしていたのでかなり時間がかかってしまった。次は最低限のプログラムにするように考える。単位ください。

そして今考えたら割合は ration じゃなくて ratio だなんて

9 作成したプログラム

```
#include <errno.h>
#include <error.h>
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define TEST_CONDITION(func, config, do_check, show_data) test_condition(func, &config, do_check, show_data, #f
#define TEST_CONDITION_WITH_DATA(func, data, do_check, show_data) test_condition_with_data(func, data, do_check

#define MIN(lhs, rhs) ((lhs) < (rhs) ? (lhs) : (rhs))
const char *PRINT_CONFIG_INDENT = "    ";
#define NEW_LINE_COUNT 10
#define ALGORITHM_COUNT 2
#define AVERAGE_COUNT 10

typedef struct {
    int *data;
    size_t size;
} Data;

typedef enum {
    RANDOM,
    PARTIAL_RANDOM,
    REVERSE,
    INPUT,
} Mode;
```

```

const char *mode_name[] = {
    "RANDOM",
    "PARTIAL_RANDOM",
    "REVERSE",
    "INPUT",
};

typedef struct {
    Mode mode;
    unsigned int seed;
    size_t data_size;
    double ration;
} Config;

Data alloc_data(const size_t size) {
    Data data;
    data.size = size;
    data.data = (int *)malloc(sizeof(int) * size);

    if (data.data == NULL) {
        fprintf(stderr, "error: in alloc_data(), failed to allocate %zu bytes.\n", sizeof(int) * size);
        fprintf(stderr, "malloc error(%s)", strerror(errno));

        exit(1);
    }

    return data;
}

void free_data(Data data) {
    free(data.data);
    data.size = 0;
    data.data = NULL;
}

// 配列用データを生成する関数
Data gen_data(const Config *config) {
    Data data;

    srand(config->seed);
    data = alloc_data(config->data_size);

    switch (config->mode) {
    case RANDOM:
        for (size_t i = 0; i < data.size; i++) {
            data.data[i] = rand();
        }
        break;

```

```

case PARTIAL_RANDOM:
    if (config->ration < 0 || config->ration > 1) {
        fprintf(stderr, "error: in gen_data(), 'config->ration' should be set between 0 and 1.\n");
        exit(1);
    }
    for (size_t i = 0; i < data.size; i++) {
        data.data[i] = i < (size_t)(data.size * config->ration) ? (int)i : rand();
    }
    break;

case REVERSE:
    for (size_t i = 0; i < data.size; i++) {
        data.data[i] = data.size - i;
    }
    break;

case INPUT:
    printf("input data\n>> ");
    for (size_t i = 0; i < config->data_size; i++) {
        scanf("%d", data.data + i);
    }
    break;

default:
    fprintf(stderr, "error: in gen_data(), 'config->mode' should be set defined in 'Mode'.\n");
    exit(1);
}

return data;
}

double get_elapsed_time() {
    static clock_t time_point = 0;
    double elapsed_time = (double)(clock() - time_point) / CLOCKS_PER_SEC;
    time_point = clock();
    return elapsed_time;
}

void print_config(const Config *config) {
    printf("%smode: %s\n", PRINT_CONFIG_INDENT, mode_name[config->mode]);
    if (config->mode != INPUT) {
        printf("%ssseed: %u\n", PRINT_CONFIG_INDENT, config->seed);
    }
    printf("%sdata_size: %zu\n", PRINT_CONFIG_INDENT, config->data_size);
    if (config->mode == PARTIAL_RANDOM) {
        printf("%sration: %lf\n", PRINT_CONFIG_INDENT, config->ration);
    }
}

```

```

static inline void swap(int *lhs, int *rhs) {
    int temp = *lhs;
    *lhs = *rhs;
    *rhs = temp;
}

void print_data(Data data) {
    if (data.size == 0) {
        return;
    }

    for (size_t i = 0; i < data.size - 1; i++) {
        printf("%d ", data.data[i]);
    }
    printf("%d\n", data.data[data.size - 1]);
}

void new_line() {
    putchar('\n');
}

typedef void (*SortAlgorithm)(Data);

int test_condition_with_data(SortAlgorithm func, Data data, bool do_check, bool show_data, const char *algorithm_name) {
    if (show_data) {
        puts("data before sorted");
        print_data(data);
    }
    puts("copying data...");
    Data data_copy = alloc_data(data.size);
    memcpy(data_copy.data, data.data, sizeof(data.data[0]) * data.size);
    puts("copied!");
    printf("start sorting with: %s\n", algorithm_name);
    puts("sorting...");
    get_elapsed_time();
    func(data_copy);
    double time = get_elapsed_time();
    puts("finished sorting!");
    printf("\tttime(ms): %d\n", (int)(time * 1000));
    new_line();

    if (show_data) {
        puts("data after sorted");
        print_data(data_copy);
    }

    if (do_check) {
        puts("checking...");
    }
}

```

```

        bool flag = true;
        for (size_t i = 1; i < data_copy.size; i++) {
            if (data_copy.data[i - 1] > data_copy.data[i]) {
                flag = false;
                break;
            }
        }

        if (flag) {
            puts("succeed!");
        }
        else {
            puts("failed to sort");
        }
    }

    new_line();

    free_data(data_copy);

    return (int)(time * 1000);
}

int test_condition(SortAlgorithm func, const Config *config, bool do_check, bool show_data, const char *algorithm_name) {
    puts("generating test case...");
    Data data = gen_data(config);
    puts("test case is generated");
    print_config(config);
    new_line();

    int value = test_condition_with_data(func, data, do_check, show_data, algorithm_name);
    free_data(data);
    return value;
}

void selection_sort(Data data) {
    int min;
    for (size_t i = 0; i < data.size; i++) {
        min = i;
        for (size_t j = i + 1; j < data.size; j++) {
            if (data.data[min] > data.data[j]) {
                min = j;
            }
        }
        swap(data.data + i, data.data + min);
    }
}

void insertion_sort(Data data) {

```

```

    for (size_t manipulating_pos = 1; manipulating_pos < data.size; manipulating_pos++) {
        size_t i;
        for (i = 0; i < manipulating_pos; i++) {
            if (data.data[i] > data.data[manipulating_pos]) {
                break;
            }
        }

        for (; i < manipulating_pos; i++) {
            swap(data.data + i, data.data + manipulating_pos);
        }
    }
}

void bubble_sort(Data data) {
    for (size_t i = 1; i < data.size; i++) {
        for (size_t j = 0; j < data.size - i; j++) {
            if (data.data[j] > data.data[j + 1]) {
                swap(data.data + j, data.data + j + 1);
            }
        }
    }
}

void quick_sort(Data data) {
    size_t left_accessor = 0;
    size_t right_accessor = data.size - 1;

    if (data.size == 2) {
        if (data.data[0] > data.data[1]) {
            swap(data.data, data.data + 1);
        }
        return;
    }
    else if (data.size <= 1) {
        return;
    }

    int pivot = data.data[data.size / 2];
    while (true) {
        while (right_accessor > 0 && data.data[right_accessor] > pivot) {
            right_accessor--;
        }
        while (left_accessor < data.size && data.data[left_accessor] < pivot) {
            left_accessor++;
        }

        if (left_accessor >= right_accessor) {
            Data left_data;

```

```

        left_data.data = data.data;
        left_data.size = right_accessor + 1;

        Data right_data;
        right_data.data = data.data + right_accessor + 1;
        right_data.size = data.size - right_accessor - 1;

        quick_sort(left_data);
        quick_sort(right_data);

        break;
    }

    swap(data.data + left_accessor, data.data + right_accessor);
    left_accessor++;
    right_accessor--;
}

}

void merge_sort(Data data) {
    if (data.size == 1) {
        return;
    }
    else if (data.size == 2) {
        if (data.data[0] > data.data[1]) {
            swap(data.data, data.data + 1);
        }
        return;
    }
}

size_t mid = data.size / 2;

Data left_data;
left_data.data = data.data;
left_data.size = mid;

Data right_data;
right_data.data = data.data + mid;
right_data.size = data.size - mid;

merge_sort(left_data);
merge_sort(right_data);

size_t left_accessor = 0;
size_t right_accessor = mid;

Data work_space = alloc_data(data.size);
size_t work_space_accessor = 0;

```

```

while (true) {
    bool left_fin = left_accessor == mid;
    bool right_fin = right_accessor == data.size;

    if (left_fin && right_fin) {
        break;
    }
    else if (left_fin) {
        while (right_accessor != data.size) {
            work_space.data[work_space_accessor++] = data.data[right_accessor++];
        }
        break;
    }
    else if (right_fin) {
        while (left_accessor != mid) {
            work_space.data[work_space_accessor++] = data.data[left_accessor++];
        }
        break;
    }
    else {
        work_space.data[work_space_accessor++] =
            data.data[left_accessor] < data.data[right_accessor] ? data.data[left_accessor++] : data.data[r
    }
}

memcpy(data.data, work_space.data, data.size * sizeof(int));
free_data(work_space);
}

void write_csv(const char *file_name, size_t time_data_count, size_t *size_data, Data time_data) {
    FILE *fp = fopen(file_name, "w");
    if (fp == NULL) {
        fprintf(stderr, "error: in write_csv(), failed to open file(file: %s).\n", file_name);
        fprintf(stderr, "fopen error(%s)", strerror(errno));

        exit(1);
    }

    for (size_t i = 0; i < time_data_count; i++) {
        fprintf(fp, "%zu,%d\n", size_data[i], time_data.data[i]);
    }

    fclose(fp);
}

void input_config(Config *config) {
    printf("input data size: ");
    scanf("%zu", &config->data_size);
}

```



```

int mode_input;
do {
    printf("input mode(0: RANDOM, 1: PARTIAL_RANDOM, 2: REVERSE, 3: INPUT): ");
    scanf("%d", &mode_input);
} while (mode_input < 0 || 3 < mode_input);
config->mode = (Mode)mode_input;

if (config->mode != INPUT) {
    printf("input seed: ");
    scanf("%u", &config->seed);
}
}

void average_tester() {
    Data data[AVERAGE_COUNT];
    Config config;

    input_config(&config);

    for (size_t i = 0; i < AVERAGE_COUNT; i++) {
        config.seed++;

        puts("generating test case...");
        data[i] = gen_data(&config);
        puts("test case is generated");
        print_config(&config);
        new_line();
    }

    Data time_data[ALGORITHM_COUNT];

    size_t time_data_count = 0;
    for (size_t size = 1; size < config.data_size; size *= 2) {
        time_data_count++;
    }

    for (size_t i = 0; i < ALGORITHM_COUNT; i++) {
        time_data[i] = alloc_data(time_data_count);
    }

    size_t *size_data = malloc(time_data_count * sizeof(size_t));
    if (size_data == NULL) {
        fprintf(stderr, "error: in main(), failed to allocate %zu bytes.\n", sizeof(size_t) * time_data_count);
        fprintf(stderr, "malloc error(%s)", strerror(errno));

        exit(1);
    }

    size_t time_data_counter = 0;

```

```

long long time_data_accessor = 0;
for (size_t size = 1; size < config.data_size; size *= 2) {

    size_data[time_data_counter] = size;
    for (size_t i = 0; i < AVERAGE_COUNT; i++) {
        time_data_accessor = 0;
        data[i].size = size;

        // time_data[time_data_accessor++].data[time_data_counter] +=
        //     TEST_CONDITION_WITH_DATA(selection_sort, *(data + i), false, false);
        // time_data[time_data_accessor++].data[time_data_counter] +=
        //     TEST_CONDITION_WITH_DATA(insertion_sort, *(data + i), false, false);
        // time_data[time_data_accessor++].data[time_data_counter] +=
        //     TEST_CONDITION_WITH_DATA(bubble_sort, *(data + i), false, false);
        time_data[time_data_accessor++].data[time_data_counter] +=
            TEST_CONDITION_WITH_DATA(quick_sort, *(data + i), false, false);
        time_data[time_data_accessor++].data[time_data_counter] +=
            TEST_CONDITION_WITH_DATA(merge_sort, *(data + i), false, false);
    }

    time_data_accessor--;
    for (; time_data_accessor >= 0; time_data_accessor--) {
        time_data[time_data_accessor].data[time_data_counter] /= AVERAGE_COUNT;
    }

    time_data_counter++;
    printf("finished sorting %f%%...\n", (double)time_data_counter / time_data_count * 100);
}

time_data_accessor = 0;
// write_csv("selection_sort.csv", time_data_count, size_data, time_data[time_data_accessor++]);
// write_csv("insertion_sort.csv", time_data_count, size_data, time_data[time_data_accessor++]);
// write_csv("bubble_sort.csv", time_data_count, size_data, time_data[time_data_accessor++]);
write_csv("quick_sort.csv", time_data_count, size_data, time_data[time_data_accessor++]);
write_csv("merge_sort.csv", time_data_count, size_data, time_data[time_data_accessor++]);

puts("-----");
puts("finished all sorting!!!!");

print_config(&config);

for (size_t i = 0; i < AVERAGE_COUNT; i++) {
    free_data(data[i]);
}
for (size_t i = 0; i < ALGORITHM_COUNT; i++) {
    free_data(time_data[i]);
}
free(size_data);
}

```

```

void normal_mode() {
    Data data;
    Config config;

    input_config(&config);

    puts("generating test case...");
    data = gen_data(&config);
    puts("test case is generated");
    print_config(&config);
    new_line();

    TEST_CONDITION_WITH_DATA(insertion_sort, data, true, true);
    TEST_CONDITION_WITH_DATA(bubble_sort, data, true, true);
}

int main() {
    average_tester();

    // Data data;
    // Config config;
    // config.data_size = 1e5;
    // config.seed = 1;
    // config.mode = RANDOM;

    // printf("input data size: ");
    // scanf("%zu", &config.data_size);

    // int mode_input;
    // do {
    //     printf("input mode(0: RANDOM, 1: PARTIAL_RANDOM, 2: REVERSE, 3: INPUT): ");
    //     scanf("%d", &mode_input);
    // } while (mode_input < 0 || 3 < mode_input);
    // config.mode = (Mode)mode_input;

    // if (config.mode != INPUT) {
    //     printf("input seed: ");
    //     scanf("%u", &config.seed);
    // }
    // puts("generating test case...");
    // data = gen_data(&config);
    // puts("test case is generated");
    // print_config(&config);
    // new_line();

    // Data time_data[ALGORITHM_COUNT];

    // size_t time_data_count = 0;

```

```

// for (size_t size = 1; size < config.data_size; size *= 2) {
//     time_data_count++;
// }

// for (size_t i = 0; i < ALGORITHM_COUNT; i++) {
//     time_data[i] = alloc_data(time_data_count);
// }

// size_t *size_data = malloc(time_data_count * sizeof(size_t));
// if (size_data == NULL) {
//     fprintf(stderr, "error: in main(), failed to allocate %zu bytes.\n", sizeof(size_t) * time_data_count);
//     fprintf(stderr, "malloc error(%s)", strerror(errno));

//     exit(1);
// }

// time_data_count = 0;
// for (size_t size = 1; size < config.data_size; size *= 2) {
//     size_t time_data_accessor = 0;
//     data.size = size;

//     size_data[time_data_count] = size;
//     time_data[time_data_accessor++].data[time_data_count] = TEST_CONDITION_WITH_DATA(selection_sort, data, true, true);
//     time_data[time_data_accessor++].data[time_data_count] = TEST_CONDITION_WITH_DATA(insertion_sort, data, true, true);
//     time_data[time_data_accessor++].data[time_data_count] = TEST_CONDITION_WITH_DATA(bubble_sort, data, true, true);
//     time_data[time_data_accessor++].data[time_data_count] = TEST_CONDITION_WITH_DATA(quick_sort, data, true, true);
//     time_data[time_data_accessor++].data[time_data_count] = TEST_CONDITION_WITH_DATA(merge_sort, data, true, true);

//     time_data_count++;
// }

// write_csv("selection_sort.csv", time_data_count, size_data, time_data[0]);
// write_csv("insertion_sort.csv", time_data_count, size_data, time_data[1]);
// write_csv("bubble_sort.csv", time_data_count, size_data, time_data[2]);
// write_csv("quick_sort.csv", time_data_count, size_data, time_data[3]);
// write_csv("merge_sort.csv", time_data_count, size_data, time_data[4]);

// puts("-----");
// puts("finished all sorting!!!!");

// print_config(&config);

// free_data(data);
// for (size_t i = 0; i < ALGORITHM_COUNT; i++) {
//     free_data(time_data[i]);
// }
// free(size_data);

// TEST_CONDITION_WITH_DATA(selection_sort, data, true, true);

```

```
// TEST_CONDITION_WITH_DATA(insertion_sort, data, true, true);  
// TEST_CONDITION_WITH_DATA(bubble_sort, data, true, true);  
// TEST_CONDITION_WITH_DATA(quick_sort, data, true, true);  
// TEST_CONDITION_WITH_DATA(merge_sort, data, true, true);  
  
return 0;  
}
```